

Practical Lab Assignment 6

Name: Abhishek Kumar

Course: B.C.A 5th Sem

Part 1 — Hoisting with var

Task 1.1: Predict, then run. Write the output in your record:

```
console.log(city);  
var city = "Haridwar";  
console.log(city);
```

Prediction

```
// undefined  
// Haridwar
```

Output

```
undefined  
Haridwar
```

Task 1.2: What does this print? Think about hoisting inside a function.

```
function showMessage() {  
  console.log(message);  
  var message = "Hello";  
  console.log(message);  
}  
  
showMessage();
```

Output

```
undefined  
Hello
```

Task 1.3 — Shadow Trap: Predict the output. Why is the first line NOT "global"?

```
var name = "global";  
function test() {  
  console.log(name);  
  var name = "local";  
}  
  
test();
```

Output

```
undefined
```

Answer: The local var name is hoisted to the top of test(), so it hides the global name before the local variable receives the value "local".

Fun Task 1.4 — "Magic Trick": Write a 4-line program that prints undefined first and then your favourite food, using only var and two console.log lines.

```
console.log(food);  
var food = "Pizza";
```

```
console.log(food);
```

Output

```
undefined  
Pizza
```

Part 2 — Function Hoisting

Task 2.1: Run this. What error appears? Write the exact message.

```
sayHi();  
var sayHi = function () {  
  console.log("Hi!");  
};
```

Expected error

```
TypeError: sayHi is not a function
```

Task 2.2: Change var to const in Task 2.1. Does the error type change? Write both error types side by side.

```
sayHi();  
const sayHi = function () {  
  console.log("Hi!");  
};  
  
// var → TypeError: sayHi is not a function  
// const → ReferenceError: Cannot access 'sayHi' before initialization
```

Task 2.3 — Sorting Table: Fill in the table. Run each one separately to check.

Code	Works before its line? Error type	
function a() {}	Yes	None
var b = function() {}	No	TypeError
const c = () => {}	No	ReferenceError
let d = function() {}	No	ReferenceError

Task 2.4 — Two Functions, Same Name: Predict, then run:

```
console.log(fnA());  
function fnA() { return "First"; }  
function fnA() { return "Second"; }
```

Output

```
Second
```

Expected: Second. The later declaration wins.

Fun Task 2.5 — "Top-Down Story": Write a small program in which the main story lines are at the TOP of the file and the helper functions are at the BOTTOM.

```
wakeUp();  
eatBreakfast();  
goToCollege();  
  
function wakeUp() {  
  console.log("Wake up!");  
}
```

```

}

function eatBreakfast() {
  console.log("Eat breakfast!");
}

function goToCollege() {
  console.log("Go to college!");
}

```

Output

```

Wake up!
Eat breakfast!
Go to college!

```

Answer: This works because function declarations are hoisted with their complete function body.

Part 3 — let, const and the Temporal Dead Zone

Task 3.1: Run the same for `const PI = 3.14;` placed after a `console.log(PI);`. Write the error message.

```

console.log(PI);
const PI = 3.14;

//ReferenceError: Cannot access 'PI' before initialization

```

Task 3.2 — typeof Surprise: Predict and run both. They are NOT the same!

```

console.log(typeof x);
var x = 5;
console.log(typeof y);
let y = 5;

//Prediction
// undefined
// ReferenceError

//Output
//undefined
// ReferenceError: Cannot access 'y' before initialization

```

Task 3.3 — Investigate: Complete the sentence: "The three kinds of mistake when using a name too early are: undefined (when ____), ReferenceError (when ____), TypeError (when ____)."

Answer: undefined (when a var variable has been hoisted but not initialized), ReferenceError (when a let/const variable is accessed in the TDZ), TypeError (when a hoisted variable is undefined but is called as if it were a function).

Fun Task 3.4 — "Error Detective": Write four mini-programs, each one causing a different result.

(a) Prints undefined

```

console.log(a);

```

```
var a = 10;
```

(b) ReferenceError

```
console.log(b);  
let b = 10;
```

(c) TypeError

```
hello();  
var hello = function() { console.log("Hi"); };
```

(d) Works because of hoisting

```
greet();  
function greet() { console.log("Hello"); }
```

Part 4 — Your First Closure (25 min)

Task 4.1: Type the counter. Call counterA 5 times and counterB 2 times. Write the outputs. Why does counterB not continue from counterA?

```
const counterA = makeCounter();  
const counterB = makeCounter();  
  
console.log(counterA());  
console.log(counterA());  
console.log(counterA());  
console.log(counterA());  
console.log(counterA());  
  
console.log(counterB());  
console.log(counterB());
```

Output

```
1  
2  
3  
4  
5  
1  
2
```

Answer: counterB does not continue from counterA because each call to makeCounter() creates a new closure with its own separate count variable.

Task 4.2: Try console.log(count); outside the function. What error? What does this prove about closure variables?

```
const counter = makeCounter();  
console.log(count);  
  
// ReferenceError: count is not defined
```

Answer: This proves that count is private to the closure and is not directly accessible from outside.

Task 4.3 — Multiplier Factory: Complete and run:

```
function makeMultiplier(n) {
  return function (x) {
    return x * n;
  };
}

const double = makeMultiplier(2);
const triple = makeMultiplier(3);
console.log(double(5), triple(5)); // 10 15
```

Output

```
10 15
```

Task 4.4: Write `makeGreeter(greeting)` so that `makeGreeter("Namaste")("Aditi")` prints `Namaste, Aditi!`.

```
function makeGreeter(greeting) {
  return function (name) {
    return greeting + ", " + name + "!";
  };
}

const greet = makeGreeter("Namaste");
console.log(greet("Aditi"));
```

Output

```
Namaste, Aditi!
```

Fun Task 4.5 — "Chai Counter": Make `makeCupCounter()` that returns a function. Each call adds one cup and returns the message "Cup number 3 of chai". Make two counters for two different friends and show they do not mix.

```
function makeCupCounter() {
  let cups = 0;
  return function () {
    cups++;
    return "Cup number " + cups + " of chai";
  };
}

const friendA = makeCupCounter();
const friendB = makeCupCounter();

console.log(friendA());
console.log(friendA());
console.log(friendA());
console.log(friendB());
console.log(friendB());
```

Output

```
Cup number 1 of chai
Cup number 2 of chai
Cup number 3 of chai
Cup number 1 of chai
Cup number 2 of chai
```

Part 5 — Private Data with Closures

Task 5.1: Type it. Try to cheat: `wallet.balance = 99999`; then `wallet.show()`. Did the money change? Why not?

```
wallet.balance = 99999;
console.log(wallet.show());
```

Output

```
120
```

Answer: The money does not change because `balance` is a private closure variable. `wallet.balance` creates a separate public property; it does not access the hidden `balance`.

Task 5.2: Add a `reset()` function that sets `balance` back to the starting amount.

```
function createWallet(start) {
  let balance = start;
  return {
    add(n) { balance += n; return balance; },
    spend(n) {
      if (n > balance) return "Insufficient balance";
      balance -= n;
      return balance;
    },
    show() { return balance; },
    reset() { balance = start; return balance; }
  };
}

const wallet = createWallet(100);
wallet.add(50);
console.log(wallet.show());
console.log(wallet.reset());
```

Output

```
150
100
```

Task 5.3 — Login Guard: Complete a closure `limiter(max)` that allows only `max` attempts.

```
function limiter(max) {
  let used = 0;
  return function () {
    if (used < max) {
      used++;
      return "Attempt " + used + " of " + max;
    } else {
      return "Locked!";
    }
  };
}

const tryLogin = limiter(3);
console.log(tryLogin());
console.log(tryLogin());
console.log(tryLogin());
console.log(tryLogin());
```

Output

```
Attempt 1 of 3
Attempt 2 of 3
Attempt 3 of 3
Locked!
```

Fun Task 5.4 — "Secret Diary": Create `createDiary()` with `write(text)` and `read()`. The diary entries must be stored in a private array.

```
function createDiary() {
  let entries = [];

  return {
    write(text) {
      entries.push(text);
    },
    read() {
      return entries;
    }
  };
}

const diary = createDiary();
diary.write("Learn closures");
diary.write("Practice JavaScript");

console.log(diary.read());
console.log(diary.entries);
```

Output

```
[ "Learn closures", "Practice JavaScript" ]
undefined
```

Part 6 — Closures in Loops

Task 6.1: Write both outputs. Explain why `var` gives `[3, 3, 3]` while `let` gives `[0, 1, 2]`.

Answer: `var` creates one shared function-scoped variable `i`, so all closures see its final value 3. `let` creates a new block-scoped binding for each loop iteration, so the closures remember 0, 1 and 2 separately.

Task 6.2: Predict first, then run this timer version. It prints after about 1 second:

```
for (var k = 1; k <= 3; k++) {
  setTimeout(() => console.log("var:", k), 1000);
}

for (let m = 1; m <= 3; m++) {
  setTimeout(() => console.log("let:", m), 1000);
}
```

Output

```
var: 4
var: 4
var: 4
let: 1
let: 2
let: 3
```

Fun Task 6.3 — "Fix the Bug": Change only ONE word in the `var` loop of Task 6.2 so it prints 1, 2, 3.

```
for (let k = 1; k <= 3; k++) {
```

```
    setTimeout(() => console.log("var:", k), 1000);  
  }
```

Output

```
var: 1  
var: 2  
var: 3
```

Part 7 — Mini Project: Smart Wallet with Login Guard

Combine hoisting and closures into one small program. Use index.html + script.js.

Requirements

1. Write createWallet(start) with add, spend, show and history. history() returns an array of text lines like "Added 200", "Spent 150".
2. Write a login guard limiter(3) as a second closure. A wallet action is allowed only if the guard says an attempt is available for the PIN check. Keep it simple: call the guard before each spend.
3. Put your main code at the TOP of the file and all functions below it as function declarations (hoisting makes this work).
4. Print a final summary using string concatenation or template literals.

Complete Program

```
const wallet = createWallet(500);  
const loginGuard = limiter(3);  
  
console.log("Starting balance:", wallet.show());  
console.log("Added:", wallet.add(200));  
  
let attempt = loginGuard();  
if (attempt !== "Locked!") {  
    console.log("Spend 150:", wallet.spend(150));  
}  
  
attempt = loginGuard();  
if (attempt !== "Locked!") {  
    console.log("Spend 1000:", wallet.spend(1000));  
}  
  
console.log("Final balance:", wallet.show());  
console.log("History:", wallet.history());  
  
function createWallet(start) {  
    let balance = start;  
    let records = [];  
  
    return {  
        add(n) {  
            balance += n;  
            records.push("Added " + n);  
            return balance;  
        },  
        spend(n) {  
            if (n > balance) return "Insufficient balance";  
            balance -= n;  
        }  
    };  
}
```



```

        records.push("Spent " + n);
        return balance;
    },
    show() {
        return balance;
    },
    history() {
        return records;
    }
};
}

function limiter(max) {
    let used = 0;
    return function () {
        if (used < max) {
            used++;
            return "Attempt " + used + " of " + max;
        }
        return "Locked!";
    };
}

```

Output

```

Starting balance: 500
Added: 700
Spend 150: 550
Spend 1000: Insufficient balance
Final balance: 550
History: [ "Added 200", "Spent 150" ]

```

Sanity Check

Start the wallet with 500. Then add(200) should give 700, spend(150) should give 550, spend(1000) should give "Insufficient balance", and show() should still be 550. history() should have 2 lines (failed spend is not recorded).

Bonus: Add a makeDiscount(percent) factory so const festive = makeDiscount(10); festive(500) gives 450.

```

function makeDiscount(percent) {
    return function (price) {
        return price - (price * percent / 100);
    };
}

const festive = makeDiscount(10);
console.log(festive(500));

```

Output

```

450

```

Part 8 — Debugging Challenge

Each snippet has a bug. Write the problem, the error or wrong output, and the fixed code.

Snippet 1

```

// Snippet 1 – runs, but prints "undefined" then 5. Make the "undefined" go
away
console.log(total);

```

```
var total = 5;
```

What Was Wrong?

The console.log runs before total receives its value. var hoisting makes total exist, but its initial value is undefined.

Fixed Code

```
var total = 5;  
console.log(total);
```

Output

```
5
```

Snippet 2

```
// Snippet 2 – TypeError. Fix it without changing "var" to "const"  
greet();  
var greet = function () { console.log("Hi"); };
```

What Was Wrong?

The variable greet is hoisted as undefined, so calling it before the assignment causes a TypeError.

Fixed Code

```
var greet = function () { console.log("Hi"); };  
greet();
```

Output

```
Hi
```

Snippet 3

```
// Snippet 3 – counter always prints 0 0  
function makeCounter() { let c = 0; return c++; }  
const next = makeCounter();  
console.log(next, next);
```

What Was Wrong?

makeCounter() returns a number, not a function. The counter state is not preserved between calls.

Fixed Code

```
function makeCounter() {  
  let c = 0;  
  return function () {  
    c++;  
    return c;  
  };  
}  
  
const next = makeCounter();  
console.log(next(), next());
```

Output

```
1 2
```

Snippet 4

```
// Snippet 4 – counter always prints 1 1 1
function makeCounter2() {
  return function () { let count = 0; count++; return count; };
}

const n = makeCounter2();
console.log(n(), n(), n());
```

What Was Wrong?

count is declared inside the returned function, so a fresh count is created every time n() is called. Nothing is remembered between calls.

Fixed Code

```
function makeCounter2() {
  let count = 0;
  return function () {
    count++;
    return count;
  };
}

const n = makeCounter2();
console.log(n(), n(), n());
```

Output

```
1 2 3
```